

Análise de Risco Sistêmico na Cadeia de Dependências do PyPI

Matheus Farias Barbosa

MC859 — Projeto em Teoria da Computação
UNICAMP

1º semestre de 2026

Resumo

Ataques à cadeia de suprimentos de software — como a porta dos fundos plantada no `xz-utils` em 2024 — mostraram que comprometer um único pacote pouco conhecido pode contaminar milhares de projetos que dependem dele indiretamente. Este trabalho modela esse problema como um grafo de dependências do ecossistema Python (PyPI) e constrói um *pipeline* de quatro fases para identificar quais pacotes representam os maiores pontos únicos de falha. A partir de dados reais do BigQuery (`deps.dev` e contagens de download do PyPI), montou-se um grafo com 4.427 pacotes e 47.568 dependências, calculou-se um score de risco e um índice de criticidade para cada nó, simulou-se a propagação de uma vulnerabilidade pelo Modelo de Cascata Independente e, por fim, confrontou-se o ranking resultante com dados reais de CVEs do OSV.dev. O modelo se mostrou um excelente classificador de priorização (*Lift* de 6 a 11 vezes acima do acaso para vulnerabilidades de alta severidade), embora não funcione como estimador contínuo de risco — uma limitação que, como se discute, decorre da própria natureza esparsa dos dados de vulnerabilidade. O resultado mais marcante: comprometer o `typing-extensions` contaminaria, em média, cerca de um terço de todo o grafo em pouco mais de três saltos.

Repositório com código fonte: [Link](#)

1 Introdução e objetivos

Praticamente nenhum software moderno é escrito do zero. Um projeto Python típico declara meia dúzia de dependências diretas, mas cada uma delas traz consigo as suas próprias dependências, e assim por diante, formando uma árvore que com frequência ultrapassa a centena de pacotes de terceiros instalados sem que o desenvolvedor sequer perceba. Essa comodidade tem um custo de segurança que ficou evidente em episódios recentes: o caso `event-stream` no npm, o pacote `ctx` sequestrado no PyPI e, sobretudo, a porta dos fundos inserida no `xz-utils` em 2024, que por muito pouco não atingiu boa parte das distribuições Linux. Em todos eles, o estrago potencial não vem da popularidade direta do pacote atacado, mas da quantidade de projetos que dependem dele *transitivamente*.

A pergunta que motiva este projeto é, portanto, estrutural: **dado o grafo de dependências do PyPI, quais pacotes concentram o maior risco sistêmico?** Ou seja, quais são os nós cuja contaminação se espalharia para a maior parte do ecossistema. Trata-se de uma pergunta de priorização — com recursos limitados de auditoria, faz sentido olhar primeiro para os pacotes mais “perigosos por posição”.

Os objetivos concretos do trabalho foram:

1. Capturar dados reais de dependências e de uso (downloads) do PyPI e montar o grafo de dependências correspondente.

2. Definir e calcular, para cada pacote, um *score de risco* $R(v)$ e um *índice de criticidade* $C(v)$ que combinem popularidade, centralidade e alcance na rede.
3. Simular a propagação de uma vulnerabilidade a partir dos pacotes mais críticos, estimando o “raio de impacto” esperado de cada um.
4. Validar o ranking produzido contra um *ground truth* independente — as vulnerabilidades reais (CVEs) registradas na base OSV.dev.

O restante do relatório segue a ordem natural do *pipeline*: captura de dados (Seção 2), construção e caracterização do grafo (Seção 3), os algoritmos das quatro fases (Seção 4), os resultados (Seção 5) e, finalmente, a análise crítica dos achados (Seção 6).

2 Captura dos dados

Toda a etapa de coleta foi feita sobre conjuntos de dados públicos hospedados no Google BigQuery, o que evitou a necessidade de *scraping*. Foram duas fontes principais, mais uma terceira usada apenas na validação.

Contagens de download (popularidade). A primeira consulta vai à tabela `bigquery-public-data.pypi.file_downloads` que registra cada download de um arquivo do PyPI. Agrupando por projeto numa janela de 30 dias, obteve-se a contagem de downloads de cada pacote. Essa contagem cumpre dois papéis: serve como critério de escolha dos pacotes-semente e, mais tarde, entra como o componente de “uso” do score de risco. Os nomes são normalizados para minúsculas, porque o PyPI trata `Requests` e `requests` como o mesmo projeto e a base de dependências usa a forma canônica.

Vale registrar uma mudança em relação ao planejamento inicial, que previa coletar a frequência de uso pela API do `pypistats.org`. Optou-se por substituí-la pelo BigQuery por dois motivos. Primeiro, a API do `pypistats` impõe forte latência e limites de requisição: baixar o histórico de milhares de pacotes, um a um, levaria tempo demais e tornaria a coleta pouco eficiente. Segundo, no começo do projeto não se sabia que o próprio BigQuery já expunha as contagens de download do PyPI na tabela `pypi.file_downloads`; ao descobrir isso no decorrer do trabalho, ficou claro que concentrar as duas fontes — uso e dependências — numa única plataforma simplificava o *pipeline* e permitia obter tudo com uma só consulta, sem *scraping* nem espera por uma API externa lenta.

Arestas de dependência. A segunda fonte é o conjunto `deps.dev` da Google (`bigquery-public-data.deps_dev`) que mantém o grafo de dependências de vários ecossistemas. Consultou-se a versão de *release* mais recente de cada pacote (usando `VersionInfo.Ordinal` para desempatar e descartando pré-lançamentos) e extraíram-se todas as arestas `PYPI → PYPI`. O resultado bruto foi bem maior do que o esperado: quase **9,8 milhões** de arestas de dependência cobrindo todo o PyPI. Trabalhar com o grafo inteiro seria caro e, na prática, pouco informativo — a imensa cauda de pacotes obscuros adicionaria ruído sem alterar a cabeça da distribuição, que é o que interessa.

Seleção das sementes e fechamento transitivo. Para recortar uma instância tratável, selecionaram-se os **5.000** pacotes mais baixados como sementes e, a partir deles, fez-se uma busca em largura (BFS) seguindo as arestas de dependência até as folhas. Em outras palavras: parte-se dos pacotes que as pessoas realmente usam e desce-se por toda a sua árvore de dependências, recolhendo todo nó alcançável e toda aresta entre nós visitados. Esse fechamento transitivo é o que define a instância de trabalho. A intuição é que um pacote só importa para a análise se algum pacote popular depender dele, direta ou indiretamente.

Ground truth de vulnerabilidades. Por fim, para a validação (Fase 4), consultou-se a API REST do **OSV.dev**, um agregador aberto de *advisories* de segurança mantido pela Google. Como são milhares de pacotes a consultar, usou-se a rota **querybatch** para descobrir baratamente quais pacotes têm *algum* aviso e só então se buscam os detalhes completos desses, evitando milhares de requisições vazias. De cada *advisory* extrai-se o score CVSS por uma estratégia de três níveis (parsing do vetor CVSS v3/v4/v2; se ausente, mapeamento do rótulo textual de severidade; se nada disso existir, descarta-se o score mas ainda se conta a vulnerabilidade).

3 Construção e caracterização da instância

O grafo é dirigido: uma aresta $u \rightarrow v$ significa “ u depende de v ”. Cada nó recebe como atributo a sua contagem de downloads, que passa por uma transformação $\log_{10}(1+x)$ seguida de normalização min-max para o intervalo $[0, 1]$. O logaritmo é essencial aqui porque a distribuição de downloads é extremamente desigual — os pacotes do topo recebem ordens de grandeza mais tráfego do que a mediana, e sem a compressão logarítmica o score de uso ficaria dominado por meia dúzia de gigantes.

A instância final tem **4.427 nós** e **47.568 arestas** (densidade $\approx 0,0024$). Três propriedades estruturais dela condicionam tudo o que vem depois e por isso merecem destaque:

1. **Distribuição de grau livre de escala (*power-law*).** O grau de entrada mediano é **1**, mas o máximo é **2.181** (o **typing-extensions**); a média fica em 10,7, puxada por essa cauda longa. A Figura 1(a) deixa o padrão evidente: pouquíssimos pacotes são dependência de quase todo mundo, enquanto a esmagadora maioria é dependência de um ou nenhum. É exatamente esse desbalanço que justifica comprimir as métricas com logaritmo antes de combiná-las.
2. **Quase-DAG.** O grafo é quase acíclico: dos 4.417 componentes fortemente conexos, apenas **4** são não-triviais e o maior tem só 8 nós (Figura 1(b)). Os poucos ciclos que existem vêm de dependências circulares legítimas (por exemplo, pacotes que se referenciam mutuamente dentro de uma mesma suíte). Isso permite tratar o grafo essencialmente como um DAG, com o cuidado de condensar os ciclos antes (ver Seção 4.1).
3. **Mundo pequeno e raso.** Apesar de milhares de nós, as cascatas simuladas na Fase 3 atingem profundidade média de apenas 3,1 a 3,5 saltos. Distâncias curtas combinadas com poucos *hubs* muito conectados são a assinatura de uma rede “mundo pequeno”.

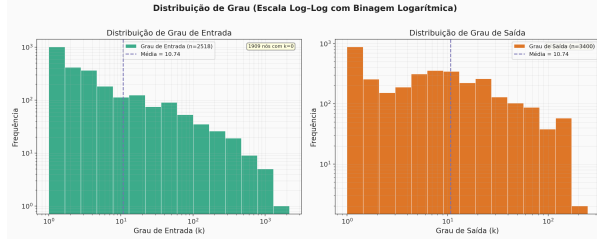
4 Algoritmos e métodos

O *pipeline* tem quatro fases que se encadeiam: cada uma lê a saída da anterior. Um ponto comum a todas as métricas é a normalização *log-minmax*: antes de qualquer combinação linear, aplica-se $\hat{x} = (\log(1+x) - \min) / (\max - \min)$. Isso põe todas as componentes na mesma escala $[0, 1]$ e, de novo, domestica as caudas pesadas.

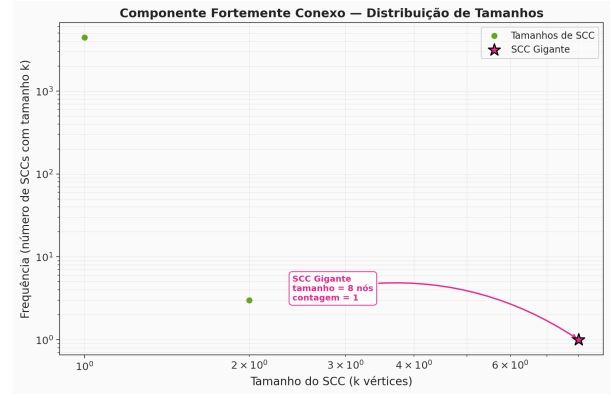
4.1 Fase 1 — score de risco $R(v)$

O score de risco combina três sinais, cada um capturando uma faceta diferente de “ser um alvo perigoso”:

$$R(v) = 0,35 \hat{d}_{\text{in}}(v) + 0,25 \hat{u}(v) + 0,40 \hat{r}(v).$$



(a) Distribuição de grau (escala log-log).



(b) Tamanho dos componentes fortemente conexos.

Figura 1: Caracterização estrutural do grafo de dependências. À esquerda, a assinatura livre de escala; à direita, a confirmação de que o grafo é quase acíclico (quase todos os SCCs têm tamanho 1).

Aqui $\hat{d}_{in}(v)$ é o grau de entrada normalizado (quantos pacotes dependem *diretamente* de v); $\hat{u}(v)$ é o uso normalizado (downloads); e $\hat{r}(v)$ é o *alcance transitivo*, isto é, quantos pacotes dependem de v direta *ou indiretamente* — o “raio de impacto” de uma falha em v . Atribuiu-se o maior peso (0,40) justamente ao alcance transitivo, porque é ele que traduz o caráter sistêmico do risco: um pacote pode ter poucos dependentes diretos e, ainda assim, sustentar meia floresta abaixo dele.

A parte algoritmicamente interessante é calcular o alcance transitivo de *todos* os nós de forma eficiente. Em vez de uma BFS reversa por nó (cara), usa-se programação dinâmica sobre o grafo em ordem topológica, representando o conjunto de ancestrais de cada nó como um *bitarray* de $|V|$ bits. A união de dois conjuntos vira um simples OU bit a bit, que o processador executa 64 bits por vez. Como o grafo tem ciclos (poucos, mas existem), primeiro ele é condensado: cada SCC vira um super-nó, roda-se a DP no DAG condensado — garantidamente acíclico — e depois mapeia-se o resultado de volta para os nós originais. O Algoritmo 1 resume o procedimento.

4.2 Fase 2 — Criticidade $C(v)$ e score combinado $S(v)$

A Fase 1 olha para o risco “intrínseco” de um pacote; a Fase 2 olha para a sua *posição estrutural* na rede. O índice de criticidade combina duas medidas clássicas de centralidade:

$$C(v) = 0,40 \hat{C}_D^{in}(v) + 0,60 \widehat{PR}(v),$$

em que $\hat{C}_D^{in}$ é a centralidade de grau de entrada e $\widehat{PR}$ é o PageRank (Brin & Page, 1998). O PageRank é especialmente adequado aqui: como ele propaga importância *contra* o sentido das arestas, um pacote ganha prestígio por ser dependência de pacotes que também são importantes — exatamente a noção de “pacote fundacional” que se procura.

Vale uma observação honesta sobre essa fórmula, porque ela não nasceu pronta — chegou-se a ela depois de tentar (e descartar) duas alternativas. A formulação inicial do projeto tinha um terceiro termo, a *betweenness* (centralidade de intermediação), pensada para capturar “gargalos” — pontes obrigatórias entre regiões do grafo —, com pesos 0,25/0,35/0,40 para grau, betweenness e PageRank. Ao medir, a ideia desmoronou. Num grafo quase acíclico quase não existem pontes obrigatórias (ou há um caminho alternativo, ou não há caminho nenhum), e o resultado foi que cerca de 97% dos nós ficaram com betweenness exatamente zero. Pior: a normalização min-max sobre um vetor quase todo nulo esticava o único nó não-trivial para 1,0, e isso bastava para alçar o *google-auth* — que, ironicamente, não tem nenhuma CVE registrada — ao segundo lugar de criticidade. Era um artefato numérico se disfarçando de achado.

Algoritmo 1 Alcance transitivo via programação dinâmica em *bitarrays*

Entrada: grafo dirigido $G = (V, E)$, com $u \rightarrow v$ significando “ u depende de v ”

Saída: $\text{reach}(v)$ = número de nós que dependem transitivamente de v

```
1: se  $G$  possui ciclos então
2:    $G' \leftarrow \text{CONDENSAÇÃO}(G)$  ▷ cada SCC vira um super-nó;  $G'$  é um DAG
3: senão
4:    $G' \leftarrow G$ 
5: fim se
6: para cada super-nó  $w \in G'$  faça
7:    $\text{bits}[w] \leftarrow$  bitarray de  $|V|$  zeros, com 1 nas posições dos membros de  $w$ 
8: fim para
9: para cada  $v$  em ordem topológica de  $G'$  faça
10:   para cada predecessor  $u$  de  $v$  em  $G'$  faça ▷  $u$  depende de  $v$ 
11:      $\text{bits}[v] \leftarrow \text{bits}[v] \text{ OU } \text{bits}[u]$ 
12:   fim para
13: fim para
14: para cada nó original  $x \in V$  faça
15:    $\text{reach}(x) \leftarrow \text{bits}[\text{super-nó de } x].\text{contagem} - 1$  ▷  $-1$  remove o próprio  $x$ 
16: fim para
17: retorne  $\text{reach}$ 
```

A primeira correção foi simples: removeu-se a *betweenness* e redistribuiu-se o seu peso entre os dois termos restantes, chegando à fórmula acima (0,40/0,60, com leve favorecimento do PageRank, que é a medida de influência global teoricamente correta para grafos dirigidos). Mas, antes de abandonar de vez a ideia de um termo estrutural, optou-se por testar uma alternativa: existiria uma centralidade *adequada* a um grafo dirigido e quase acíclico? O candidato natural era a **centralidade de Katz** (Katz, 1953), bem definida justamente onde a centralidade de autovetor degenera — em DAGs. Implementou-se um modo experimental que troca a *betweenness* por Katz (resolvida por fatoração direta, sem risco de não-convergência), mantendo os pesos originais.

Não vingou, por dois motivos. Primeiro, não melhorou nada. Como a validação padrão da Fase 4 mede só $R(v)$, foi preciso comparar as variantes à parte, validando a criticidade *pura* $C(v)$ diretamente contra o *ground truth* do OSV; o Katz ficou igual ou *ligeiramente pior* que a versão sem termo estrutural (Tabela 1). Segundo, e mais interessante: num DAG, a centralidade de Katz se reduz a uma contagem atenuada de ancestrais — ou seja, ela re-expressa boa parte do *alcance transitivo* que o score $R(v)$ já carrega. Seria contar a mesma coisa duas vezes. A lição é que um caminho que não leva a lugar nenhum, quando documentado, também é resultado: a versão atual fica com $C(v) = 0,40 \hat{C}_D^{\text{in}} + 0,60 \widehat{PR}$, e os modos **betweenness** e **katz** permanecem disponíveis apenas por trás de uma chave de configuração, para reprodutibilidade.

Tabela 1: Lift@ k (vulnerabilidade *high*) da criticidade pura $C(v)$ sob as três variantes do termo estrutural. A aparente vantagem da *betweenness* em $k = 25$ e 50 é de um acerto a mais — dentro do ruído amostral — e não compensa nem o artefato do **google-auth** nem o custo $O(V \cdot E)$. O Katz não supera a versão sem termo.

k	betweenness (falha)	sem termo (atual)	Katz (experimental)
10	10,6	10,6	10,6
25	8,5	7,7	6,8
50	7,7	6,8	6,4
100	6,6	6,4	6,4

Algoritmo 2 Simulação de cascata (ICM) com estimativa de Monte Carlo

Entrada: grafo reverso G^T com probabilidades $p(v \rightarrow u)$ pré-computadas; semente s ; N repetições
Saída: raio de impacto esperado $\mathbb{E}[B(s)]$

```
1: para  $i \leftarrow 1$  até  $N$  faça
2:   infectados  $\leftarrow \{s\}$ ; fronteira  $\leftarrow \{s\}$ 
3:   enquanto fronteira  $\neq \emptyset$  faça
4:     nova  $\leftarrow \emptyset$ 
5:     para cada nó  $w \in$  fronteira faça
6:       para cada vizinho  $u$  de  $w$  em  $G^T$  com  $u \notin$  infectados faça
7:         se ALEATÓRIO(0, 1)  $< p(w \rightarrow u)$  então
8:           adicione  $u$  a infectados e a nova
9:         fim se
10:      fim para
11:    fim para
12:    fronteira  $\leftarrow$  nova
13:  fim enquanto
14:   $B_i \leftarrow |\text{infectados}|$ 
15: fim para
16: retorne  $\frac{1}{N} \sum_{i=1}^N B_i$ 
```

O score final que ordena os pacotes mistura risco e criticidade em partes iguais:

$$S(v) = 0,5 R(v) + 0,5 C(v).$$

4.3 Fase 3 — Simulação de cascata (Modelo de Cascata Independente)

Os scores anteriores são estáticos. Para responder “*o que aconteceria de fato se este pacote fosse comprometido?*” recorre-se a uma simulação estocástica: o **Modelo de Cascata Independente** (ICM), emprestado da teoria de maximização de influência em redes sociais (Kempe, Kleinberg & Tardos, 2003).

A simulação roda sobre o grafo *reverso*: se u depende de v , uma vulnerabilidade em v “infecta” u (que executa o código de v), então a contaminação flui de v para u . Cada aresta tem uma probabilidade de propagação

$$p(v \rightarrow u) = 0,01 + 0,99 \cdot R(v) \cdot \hat{u}(u),$$

ou seja: um piso de 1% (até dependências raras podem propagar um zero-day crítico), escalado pelo risco da fonte e pelo uso do alvo. Partindo de um pacote-semente, sorteia-se a propagação aresta a aresta em ondas, até nenhum novo nó ser infectado. Como o processo é aleatório, repete-se $N = 1000$ vezes (Monte Carlo) e toma-se a média do tamanho final do conjunto infectado — o *raio de impacto esperado* $\mathbb{E}[B(v)]$ — junto com um intervalo de confiança de 95%. O Algoritmo 2 descreve uma rodada e a agregação.

4.4 Fase 4 — Validação contra dados reais (OSV.dev)

De nada adianta um ranking bonito se ele não bate com a realidade. A Fase 4 testa se os pacotes que o modelo aponta como mais arriscados são, de fato, os que mais acumulam vulnerabilidades reais. A metodologia separa dois grupos: **Grupo A**, os 1.000 pacotes de maior $R(v)$; e **Grupo B**, 1.000 pacotes sorteados aleatoriamente do restante, que funciona como grupo de controle para estimar a taxa-base de vulnerabilidade do ecossistema. As métricas usadas foram:

- **Correlação** (Spearman e Kendall, calculadas no Grupo B para evitar viés de restrição de amplitude), medindo se mais risco anda junto com mais vulnerabilidades.
- **Correlação parcial** controlando popularidade, para isolar se a *topologia* prediz vulnerabilidades independentemente do número de downloads.
- **Precision@ k e Lift@ k** para $k \in \{10, 25, 50, 100\}$: das k cabeças do ranking, que fração é de fato vulnerável, e quantas vezes isso supera a taxa-base (o *lift*).
- **Teste U de Mann–Whitney**, não-paramétrico, comparando a distribuição de vulnerabilidades dos dois grupos.
- **Robustez**: análise de sensibilidade ao limiar de CVSS e intervalos de confiança por *bootstrap* (10.000 reamostragens).

5 Resultados

O *pipeline* completo roda em poucos minutos. Os resultados são apresentados na ordem das fases.

Fase 1 — quem são os pacotes mais arriscados. O topo do ranking de $R(v)$ é, tranquilizadamente, uma lista de quem qualquer pessoa que programa em Python reconhece de imediato: infraestrutura de tipagem, HTTP, TLS e *parsing*. A Tabela 2 mostra os dez primeiros. Que o modelo recupere espontaneamente esses “pilares” do ecossistema é um primeiro sinal de sanidade — são justamente os pontos únicos de falha que a proposta queria achar.

Tabela 2: Top-10 por score de risco $R(v)$ (Fase 1).

#	Pacote	Grau entrada	Alcance (r)	$R(v)$
1	typing-extensions	2.181	2.188	0,984
2	idna	1.165	1.171	0,925
3	certifi	1.100	1.102	0,919
4	urllib3	1.001	1.003	0,910
5	requests	906	908	0,900
6	charset-normalizer	920	922	0,899
7	packaging	777	782	0,888
8	six	693	696	0,865
9	python-dateutil	580	583	0,849
10	pyyaml	509	509	0,835

Fase 2 — criticidade e score combinado. Ao incorporar PageRank e grau, a ordenação por $S(v)$ permanece muito parecida, com o **typing-extensions** isolado na liderança (score combinado 0,99, contra $\approx 0,60$ do segundo colocado). Esse abismo entre o primeiro e os demais já antecipa o achado da fase seguinte: há um nó claramente em uma categoria própria de importância estrutural.

Fase 3 — o raio de impacto. Este é o resultado considerado mais relevante do trabalho (Tabela 3). Comprometer um único pacote, o **typing-extensions**, contaminaria em média **1.507 pacotes — cerca de 34% de todo o grafo** — em pouco mais de três saltos. Os cinco pacotes seguintes sozinhos já colocam entre 12% e 20% do ecossistema em risco cada um. É a confirmação quantitativa, no recorte do PyPI, do fenômeno que Zimmermann et al. (2019) batizaram de “mundo pequeno com altos riscos”.

Tabela 3: Raio de impacto esperado por semente (Fase 3, ICM com $N = 1000$).

Semente	$\mathbb{E}[B]$	% do grafo	Profundidade média
typing-extensions	1.507	~34%	3,09
idna	881	~20%	3,11
certifi	821	~18%	3,15
urllib3	715	~16%	3,18
charset-normalizer	668	~15%	3,27
requests	521	~12%	3,28

Fase 4 — o ranking bate com a realidade? Consultou-se o OSV.dev para os 2.000 pacotes dos dois grupos; **253 (12,7%)** tinham ao menos um *advisory* e 197 tinham vulnerabilidade de alta severidade ($CVSS \geq 7$). A taxa-base de vulnerabilidade *high* no grupo de controle é de apenas 4,7%. Contra esse pano de fundo, os números de enriquecimento são expressivos (Tabela 4): entre os 10 pacotes mais arriscados, metade já teve uma vulnerabilidade grave — um *lift* de **10,6×** sobre o acaso — e o efeito se mantém forte até $k = 100$. O teste de Mann–Whitney separa os dois grupos com significância esmagadora ($p \approx 10^{-20}$). Por outro lado, e isso merece honestidade, as correlações contínuas (Spearman, Kendall e a correlação parcial) *falharam* — esse ponto, mais interessante do que parece, é retomado na análise.

Tabela 4: Resultados da validação contra o OSV.dev (Fase 4). Critério de aprovação entre parênteses.

Teste	Valor	Veredito
Spearman ρ (Grupo B)	-0,022 ($p = 0,49$)	reprovado ($\rho > 0,3$)
Kendall τ (Grupo B)	-0,017 ($p = 0,50$)	reprovado
Correlação parcial (controla uso)	-0,085 ($p = 0,007$)	reprovado ($\rho > 0,15$)
Mann–Whitney U (A vs. B)	$p \approx 9,4 \times 10^{-20}$	aprovado
Precision@10 (<i>high</i>)	50%	—
Lift@10 / @25 / @50 / @100	10,6× / 7,7× / 6,8× / 5,7×	aprovado (@25>3)
Sensibilidade ao limiar CVSS	4/4 limiares com Lift@25>3	robusto

6 Análise, interpretação e descobertas

À primeira vista, a Fase 4 parece contraditória: como o modelo pode “passar arrasador” em *lift* e Mann–Whitney e, ao mesmo tempo, “falhar” na correlação de Spearman? A resposta — e essa foi a descoberta conceitual mais valiosa do projeto — é que os dois grupos de testes medem coisas diferentes, e o contraste entre eles diz muito sobre a natureza do problema.

O modelo é um classificador de cauda, não um termômetro contínuo. A variável-alvo (número de vulnerabilidades por pacote) é *fortemente inflada em zeros*: com taxa-base de 6%, cerca de 94% dos pacotes do grupo de controle têm exatamente zero *advisories*, e a mediana é 0 nos dois grupos. O coeficiente de Spearman calculado sobre um vetor que é 94% empate em zero converge para perto de zero por construção — ele está medindo ruído de ordenação dentro de uma massa de zeros, não ausência de sinal. O sinal *existe e é forte*, mas vive na cauda extrema da distribuição, que é precisamente o que Precision@ k , Lift@ k e Mann–Whitney capturam. Em termos práticos: o modelo não serve para dizer “o pacote X é 1,3 vez mais arriscado que o Y”, mas é excelente para responder “por onde a equipe de segurança deve começar a auditar” — e é exatamente para isso que ele foi pensado.

E, afinal, o que significa “mais vulnerável”? Há ainda uma dificuldade conceitual, anterior aos dados, em querer um termômetro contínuo. Dizer que “o pacote X é mais vulnerável que o Y” pressupõe que vulnerabilidade seja uma grandeza bem-ordenada, e ela não é: depende de uma porção de fatores que um modelo baseado em estrutura e popularidade sequer enxerga. Uma vulnerabilidade exposta por um pacote pode morar num trecho de código que os seus dependentes nunca chegam a chamar; ou pode ser *neutralizada* na implementação de quem o usa — por exemplo, um dependente que valida a entrada antes de repassá-la, que isola a chamada, ou que já fixou a versão corrigida —, de modo que, na prática, não há problema algum a jusante. Pesam também a severidade real da falha, a existência de um *exploit* público, se o caminho vulnerável é de fato alcançável em produção e a velocidade com que a correção se propaga pela cadeia. Nada disso se reduz ao número de downloads ou à posição no grafo. Ou seja: insistir num score contínuo e absoluto de “vulnerabilidade” seria prometer uma precisão que o problema simplesmente não comporta. É mais um motivo para tratar o modelo pelo que ele faz bem — ordenar prioridades de auditoria — em vez de exigir dele uma probabilidade que dependeria de informação que nenhum grafo de dependências, sozinho, contém.

Um resultado negativo honesto: topologia não é independente de popularidade. A correlação parcial controlando o número de downloads deu $-0,085$ — estatisticamente significativa, porém negativa. Pela hipótese “forte” do projeto, esperava-se que a estrutura do grafo (grau + alcance) predissesse vulnerabilidades mesmo depois de descontado o efeito de popularidade. Os dados não sustentam isso. A leitura mais provável é a **Lei de Linus** (o famoso “*given enough eyeballs, all bugs are shallow*”): pacotes muito populares recebem mais escrutínio e, por isso, têm mais CVEs *descobertas* — não necessariamente são mais inseguros. O sinal preditivo, neste recorte, mora na popularidade, e a topologia residual não acrescenta poder.

O *ground truth* é esparso e atrasado. Há ainda uma ressalva importante para interpretar os 12,7% de pacotes com *advisory*: ausência de CVE não é prova de segurança. Alfadel et al. (2023) mostram que vulnerabilidades em pacotes Python levam, em média, mais de três anos para serem descobertas. Ou seja, boa parte dos “zeros” do *ground truth* são provavelmente vulnerabilidades ainda não descobertas. Isso põe um teto desconhecido em qualquer correlação contínua e reforça por que o enriquecimento (priorização) é a métrica certa para este tipo de dado.

Onde o trabalho conversa com a literatura. Os números de *lift* de 6 a 11 vezes estão na mesma direção dos resultados de priorização por métricas de Shin et al. (2011) e Zimmermann & Nagappan (2008), ainda que os valores absolutos não sejam diretamente comparáveis (granularidades e taxas-base distintas). O raio de impacto de $\sim 34\%$ num único pacote é o mesmo fenômeno estrutural descrito por Zimmermann et al. (2019) para o npm, em escala proporcional ao subgrafo. E a propagação transitiva em poucos saltos ecoa Decan et al. (2018).

7 Conclusão

Este trabalho apresentou um *pipeline* que parte de dados públicos do PyPI, construído de ponta a ponta, monta um grafo de 4.427 pacotes, atribui a cada um deles um score de risco e de criticidade, simula a propagação de uma vulnerabilidade e valida o ranking contra CVEs reais. O modelo cumpre bem o objetivo para o qual foi desenhado — priorizar pacotes para auditoria —, identificando os pontos únicos de falha do ecossistema com uma taxa de acerto de 6 a 11 vezes acima do acaso para vulnerabilidades graves. A simulação de cascata quantificou de forma concreta o risco sistêmico: o comprometimento do *typing-extensions* alcançaria, em expectativa, um terço de todo o grafo.

Tão importante quanto os acertos foram as limitações que o experimento expôs: o modelo é um classificador de cabeça de cauda, e não um estimador contínuo; e, neste recorte de dados, a estrutura do grafo não se mostrou um preditor de vulnerabilidades independente da popularidade. Longe de enfraquecer o trabalho, esses resultados negativos — somados ao reconhecimento de que a betweenness é inadequada num grafo quase acíclico — foram o que mais ensinou sobre o problema. Como direções futuras, valeria ampliar o grafo para incluir a verdadeira cauda do PyPI (dando variância de desfecho às correlações), calibrar os pesos e as probabilidades contra dados históricos e adotar modelos estatísticos próprios para dados inflados em zero.

Referências

- [1] Brin, S.; Page, L. (1998). *The Anatomy of a Large-Scale Hypertextual Web Search Engine*. Computer Networks, 30(1–7), 107–117.
- [2] Brandes, U. (2001). *A Faster Algorithm for Betweenness Centrality*. Journal of Mathematical Sociology, 25(2), 163–177.
- [3] Katz, L. (1953). *A new status index derived from sociometric analysis*. Psychometrika, 18(1), 39–43.
- [4] Kempe, D.; Kleinberg, J.; Tardos, É. (2003). *Maximizing the Spread of Influence through a Social Network*. KDD 2003.
- [5] Zimmermann, M.; Staicu, C.-A.; Tenny, C.; Pradel, M. (2019). *Small World with High Risks: A Study of Security Threats in the npm Ecosystem*. USENIX Security 2019.
- [6] Decan, A.; Mens, T.; Constantinou, E. (2018). *On the Impact of Security Vulnerabilities in the npm Package Dependency Network*. MSR 2018.
- [7] Alfadel, M.; Costa, D. E.; Shihab, E. (2023). *Empirical analysis of security vulnerabilities in Python packages*. Empirical Software Engineering, 28, 59.
- [8] Zimmermann, T.; Nagappan, N. (2008). *Predicting Defects using Network Analysis on Dependency Graphs*. ICSE 2008.
- [9] Shin, Y.; Meneely, A.; Williams, L.; Osborne, J. (2011). *Evaluating Complexity, Code Churn, and Developer Activity Metrics as Indicators of Software Vulnerabilities*. IEEE TSE, 37(6).
- [10] FIRST.Org (2019). *Common Vulnerability Scoring System (CVSS) v3.1 Specification*.
- [11] Google Open Source (2021–). *OSV.dev — Open Source Vulnerabilities database*. <https://osv.dev>
- [12] Google (2021–). *deps.dev / BigQuery public datasets (deps_dev_v1, pypi.file_downloads)*.